AUTOMATA THEORY

Theoretical Computer Science (TCS)

Comprehensive Module-Wise Syllabus Revision & Exam Guide

Complete Academic Reference Book & Syllabus Coverage

Designed for Information Technology & Computer Engineering Undergraduates
Grounded in Mumbai University Syllabus (TCET / IT Curriculum Standards)

Key Course Highlights Covered in Detail

- **Module 1:** Introduction, Alphabets, Languages, Formal Definitions of FA, DFA, NFA, Moore & Mealy Machines.
- **Module 2:** Regular Expressions, Arden's Theorem, NFA to DFA Conversion, DFA Minimization (Box Method).
- **Module 3:** Regular Languages, Pumping Lemma for Regular Languages, Closure Properties, Myhill-Nerode Theorem.
- **Module 4:** Context-Free Grammars (CFG), Parse Trees, Ambiguity Removal, Simplification, CNF & GNF Normal Forms.
- **Module 5:** Pushdown Automata (PDA), Instantaneous Descriptions, CFG $\leftrightarrow$ PDA Equivalence, DPDA vs NPDA.
- **Module 6:** Turing Machines (TM), Halting Problem, Undecidability, Rice's Theorem, Post Correspondence Problem (PCP).

Academic Year / Semester Edition
Document Version: 2.0 (Expanded Master Reference Guide)
Generated for Comprehensive Study & Exam Preparation

Contents

Course Syllabus & Pedagogical Roadmap **2**

1 Module 1: Introduction & Mathematical Preliminaries **3**
1.1 Basic Mathematical Concepts . . . 3
1.1.1 Alphabets, Strings, and Languages . . . 3
1.1.2 Operations on Languages . . . 3
1.2 Overview of Automata and Formal Languages . . . 3
1.3 Deterministic Finite Automata (DFA) . . . 3
1.3.1 Formal Definition of DFA . . . 3
1.3.2 Extended Transition Function ($\hat{\delta}$) . . . 4
1.3.3 Language Accepted by a DFA . . . 4
1.4 Nondeterministic Finite Automata (NFA) . . . 5
1.4.1 Formal Definition of NFA . . . 5
1.5 Finite Automata with Output (Moore and Mealy Machines) . . . 5
1.5.1 Moore Machine . . . 5
1.5.2 Mealy Machine . . . 6

2 Module 2: Finite Automata & Regular Expressions **7**
2.1 ϵ-NFA and Elimination of Null Transitions . . . 7
2.1.1 ϵ-Closure (E-closure) . . . 7
2.2 Subset Construction Algorithm (NFA to DFA Conversion) . . . 7
2.3 DFA Minimization (Myhill-Nerode / Box-Table Filling Method) . . . 7
2.4 Regular Expressions (RE) . . . 8
2.4.1 Formal Base Inductive Definition . . . 8
2.4.2 Algebraic Laws for Regular Expressions . . . 8
2.5 Arden's Theorem . . . 9

3 Module 3: Regular Languages & Pumping Lemma **10**
3.1 Proving Languages Non-Regular . . . 10
3.2 Pumping Lemma for Regular Languages . . . 10
3.2.1 Proof Strategy by Contradiction (Adversarial Structure) . . . 10
3.3 Closure Properties of Regular Languages . . . 11
3.4 Myhill-Nerode Theorem . . . 11

4 Module 4: Context-Free Grammars (CFG) & Languages **12**
4.1 Formal Definition of Context-Free Grammar (CFG) . . . 12
4.2 Derivations and Parse Trees . . . 12
4.2.1 Derivation Types . . . 12
4.2.2 Parse Tree (Derivation Tree) . . . 12
4.3 Ambiguity in Grammars . . . 12
4.4 Simplification of Context-Free Grammars . . . 13
4.5 Normal Forms for CFGs . . . 13
4.5.1 Chomsky Normal Form (CNF) . . . 13
4.5.2 Greibach Normal Form (GNF) . . . 13
4.6 Pumping Lemma for Context-Free Languages (CFL) . . . 14

5 Module 5: Pushdown Automata (PDA) 15
5.1 Pushdown Automata Architecture . . . 15
5.1.1 Formal Definition . . . 15
5.2 Instantaneous Description (ID) . . . 15
5.2.1 Transition Notation . . . 15
5.3 Language Acceptance Criteria . . . 15
5.4 Deterministic Pushdown Automata (DPDA) . . . 16
5.5 Conversion of CFG to PDA . . . 16

6 Module 6: Turing Machines (TM) & Undecidability 17
6.1 Turing Machine Model . . . 17
6.1.1 Formal Definition . . . 17
6.2 Variants of Turing Machines . . . 17
6.3 Church-Turing Thesis . . . 18
6.4 Halting Problem of Turing Machines . . . 18
6.5 Rice's Theorem . . . 18
6.6 Post Correspondence Problem (PCP) . . . 18
6.7 Recursive and Recursively Enumerable Languages . . . 19

7 University Exam Revision Solved Question Bank 20
7.1 Short Answer Concept Clarification Questions (5 Marks Each) . . . 20
7.2 Long Analytical Construction Problems (10 Marks Each) . . . 20

8 Summary Formula Cheat Sheet 22

Course Syllabus Pedagogical Roadmap

The study of **Automata Theory and Formal Languages** forms the theoretical backbone of Computer Science and Information Technology. This subject investigates what can and cannot be computed using abstract mathematical machines. The curriculum is structured into six progressive modules that map directly to the Chomsky Hierarchy of languages and computational power.

Table 1: **Automata Theory Module-Wise Syllabus Map**

Module	Core Topic Area	Key Abstract Model	Weight / Hrs
1	Mathematical Foundations & Finite Automata	DFA, NFA, Moore & Mealy Machines	05 Hrs
2	Regular Expressions & Language Equivalences	Thompson's NFA, Arden's RE, Box Minimization	09 Hrs
3	Regular Languages & Non-Regularity Proofs	Pumping Lemma for RL, Closure Properties	06 Hrs
4	Context-Free Grammars & Normal Forms	CFG, Derivation Trees, CNF, GNF, Pumping Lemma	08 Hrs
5	Pushdown Automata & Context-Free Languages	NPDA, DPDA, Instantaneous Descriptions	08 Hrs
6	Turing Machines & Computability Limits	Single/Multi-Tape TM, Halting Problem, PCP	09 Hrs

The Chomsky Hierarchy Summary

- **Type-3 (Regular Languages):** Recognized by *Finite Automata* (DFA/NFA). Generated by Regular Grammars ($A \to aB$ or $A \to a$).
- **Type-2 (Context-Free Languages):** Recognized by *Pushdown Automata* (PDA). Generated by Context-Free Grammars ($A \to \alpha$).
- **Type-1 (Context-Sensitive Languages):** Recognized by *Linear Bounded Automata* (LBA). Generated by Context-Sensitive Grammars ($\alpha A \beta \to \alpha \gamma \beta$).
- **Type-0 (Recursively Enumerable Languages):** Recognized by *Turing Machines* (TM). Generated by Unrestricted Grammars ($\alpha \to \beta$).

1 Module 1: Introduction & Mathematical Preliminaries

1.1 Basic Mathematical Concepts

Automata theory relies on precise mathematical language defined over symbols, strings, and sets.

1.1.1 Alphabets, Strings, and Languages

- **Alphabet (Σ):** A finite, non-empty set of symbols.
  - Binary Alphabet: $\Sigma = \{0, 1\}$
  - English Alphabet: $\Sigma = \{a, b, c, \ldots, z\}$
- **String (or Word):** A finite sequence of symbols chosen from an alphabet Σ.
  - Example: For $\Sigma = \{0, 1\}$, $w = 01101$ is a string of length $|w| = 5$.
  - **Empty String (ϵ or λ):** A string containing zero symbols, with length $|\epsilon| = 0$.
- **Powers of an Alphabet:**
  - $\Sigma^0 = \{\epsilon\}$
  - $\Sigma^1 = \{0, 1\}$
  - $\Sigma^2 = \{00, 01, 10, 11\}$
  - $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$ (Kleene Closure / Set of all strings of finite length).
  - $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \cdots = \Sigma^* \setminus \{\epsilon\}$ (Positive Closure).
- **Language (L):** Any set of strings chosen from Σ^*, i.e., $L \subseteq \Sigma^*$.
  - $L_1 = \{w \in \{0, 1\}^* \mid |w| \text{ is even}\}$
  - $L_2 = \{a^n b^n \mid n \geq 1\}$

1.1.2 Operations on Languages

Let L_1 and L_2 be two languages over Σ:

1. **Union:** $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ or } w \in L_2\}$
2. **Intersection:** $L_1 \cap L_2 = \{w \mid w \in L_1 \text{ and } w \in L_2\}$
3. **Concatenation:** $L_1 L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$
4. **Complement:** $\overline{L_1} = \Sigma^* \setminus L_1$
5. **Reversal:** $L^R = \{w^R \mid w \in L\}$
6. **Kleene Star:** $L^* = \bigcup_{i=0}^{\infty} L^i$, where $L^0 = \{\epsilon\}$ and $L^i = LL^{i-1}$.

1.2 Overview of Automata and Formal Languages

An **automaton** is an abstract digital computer that receives discrete input, changes its internal states according to transition rules, and yields outputs or decisions (Accept/Reject).

1.3 Deterministic Finite Automata (DFA)

1.3.1 Formal Definition of DFA

A **Deterministic Finite Automaton (DFA)** is formally defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F) \tag{1}$$

where:

- Q is a finite, non-empty set of internal states.

Table 2: **Comparison of Automata Models**

Automaton	Memory Model	Language Recognized	Boundary / Limits
Finite Automaton (FA)	Finite States (No extra memory)	Regular Languages	Cannot count unbounded symbols ($a^n b^n$).
Pushdown Automaton (PDA)	Single Stack (LIFO Memory)	Context-Free Languages	Limited to 1 stack (cannot recognize $a^n b^n c^n$).
Linear Bounded Automaton	Bounded Tape ($\leq k \cdot \lvert w \rvert$)	Context-Sensitive	Solves context-dependent problems.
Turing Machine (TM)	Unbounded Read/Write Tape	Recursively Enumerable	General model of computation.

- Σ is a finite input alphabet.
- $\delta : Q \times \Sigma \to Q$ is the total transition function mapping a state and an input symbol to a unique next state.
- $q_0 \in Q$ is the initial (start) state.
- $F \subseteq Q$ is the set of final (accepting) states.

1.3.2 Extended Transition Function ($\hat{\delta}$)

To describe machine behavior over strings rather than single symbols, we extend δ to $\hat{\delta} : Q \times \Sigma^* \to Q$ recursively:

$$\hat{\delta}(q, \epsilon) = q \tag{2}$$

$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a) \quad \text{for } w \in \Sigma^*, a \in \Sigma \tag{3}$$

1.3.3 Language Accepted by a DFA

A language $L(M)$ accepted by a DFA M is defined as:

$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\} \tag{4}$$

DFA Construction Example 1: Strings Containing Substring "101"

Problem Statement: Design a DFA over $\Sigma = \{0, 1\}$ that accepts all strings containing 101 as a substring.

Solution Formulation: Let $Q = \{q_0, q_1, q_2, q_3\}$, q_0 = start state, $F = \{q_3\}$. State semantics:

- q_0: Initial state (no part of "101" seen yet).
- q_1: "1" has been read.
- q_2: "10" has been read.
- q_3: "101" has been seen (Dead/Trap Accept State).

Transition Table:

State Q	Input 0	Input 1
$\to q_0$	q_0	q_1
q_1	q_2	q_1
q_2	q_0	q_3
$*q_3$	q_3	q_3

DFA Construction Example 2: Strings with Modulo Arithmetic

Problem Statement: Design a DFA over $\Sigma = \{a, b\}$ accepting strings where the length $|w| \pmod 3 = 0$.
Solution: Let $Q = \{q_0, q_1, q_2\}$, q_0 = initial state, $F = \{q_0\}$.

- $\delta(q_0, a) = q_1, \quad \delta(q_0, b) = q_1$
- $\delta(q_1, a) = q_2, \quad \delta(q_1, b) = q_2$
- $\delta(q_2, a) = q_0, \quad \delta(q_2, b) = q_0$

1.4 Nondeterministic Finite Automata (NFA)

1.4.1 Formal Definition of NFA

A **Nondeterministic Finite Automaton (NFA)** is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F) \tag{5}$$

where δ is a transition function that maps a state and symbol to a **power set** of states:

$$\delta : Q \times \Sigma \to 2^Q \tag{6}$$

Unlike DFAs, an NFA can have zero, one, or multiple transitions from a single state on a given input symbol. A string w is accepted if *at least one* path leads to a state in F.

Table 3: **Key Differences: DFA vs NFA**

Deterministic Finite Automaton (DFA)	**Nondeterministic Finite Automaton (NFA)**
Transition function $\delta : Q \times \Sigma \to Q$.	Transition function $\delta : Q \times \Sigma \to 2^Q$.
For every state and input symbol, exactly one transition exists.	Zero, one, or multiple transitions allowed for an input symbol.
No ϵ-transitions permitted.	Can permit ϵ-transitions (ϵ-NFA).
Execution path for any string is strictly unique.	Multiple execution paths explored simultaneously.
Easier to implement in software (hardware/parsers).	Easier to design conceptually for complex regular languages.
Time complexity per symbol: $O(1)$.	Time complexity per symbol: $O(\|Q\|^2)$ if simulated directly.

1.5 Finite Automata with Output (Moore and Mealy Machines)

Finite automata can be augmented with an output alphabet to perform transductions (e.g., binary addition, 2's complement).

1.5.1 Moore Machine

A **Moore Machine** is a 6-tuple:

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0) \tag{7}$$

where:

- Δ is the finite output alphabet.

- $\lambda : Q \to \Delta$ is the output function mapping **each state** to an output symbol.

Output Length Property: For an input string of length n, a Moore machine produces an output string of length $n+1$ because it emits $\lambda(q_0)$ prior to reading any input.

1.5.2 Mealy Machine

A **Mealy Machine** is a 6-tuple:

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0) \tag{8}$$

where λ is the output function mapping **state-transition pairs** to output symbols:

$$\lambda : Q \times \Sigma \to \Delta \tag{9}$$

Output Length Property: For an input string of length n, a Mealy machine produces an output string of length n.

Moore Machine Solved Example: 1's Complement Generator

Problem: Design a Moore machine that outputs the 1's complement of an input binary string.
State Definitions: Let $Q = \{q_0, q_1\}$, $\Sigma = \{0, 1\}$, $\Delta = \{0, 1\}$, initial state $= q_0$.

- q_0: Output 0 (Associated with receiving symbol '1').
- q_1: Output 1 (Associated with receiving symbol '0').

Transition & Output Table:

Present State	Next State ($a=0$)	Next State ($a=1$)	Output $\lambda(\mathbf{q})$
$\to q_0$	q_1	q_0	0
q_1	q_1	q_0	1

Mealy Machine Solved Example: 2's Complement Generator

Problem: Design a Mealy machine to find the 2's complement of a binary string read from Right to Left (LSB to MSB).
Logic: Scan from LSB to MSB. Retain all initial 0s as 0 and the first 1 as 1. Once the first 1 has passed, flip all subsequent bits ($0 \to 1$ and $1 \to 0$).
States:

- q_0: State before encountering the first 1.
- q_1: State after encountering the first 1.

Transitions & Outputs:

Present State	Next State ($a=0$)	Output ($a=0$)	Next State ($a=1$)	Output ($a=1$)
$\to q_0$	q_0	0	q_1	1
q_1	q_1	1	q_1	0

2 Module 2: Finite Automata & Regular Expressions

2.1 ϵ-NFA and Elimination of Null Transitions

An **ϵ-NFA** permits state transitions without consuming any input symbol, modeled as $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^Q$.

2.1.1 ϵ-Closure (E-closure)

For any state $q \in Q$, E-closure(q) is the set of all states reachable from q by taking zero or more ϵ-transitions.

$$\text{E-closure}(q) = \{p \in Q \mid q \xrightarrow{\epsilon^*} p\} \tag{10}$$

Algorithm: Converting ϵ-NFA to NFA without ϵ-Transitions

Given an ϵ-NFA $M = (Q, \Sigma, \delta, q_0, F)$, construct an equivalent NFA $M' = (Q, \Sigma, \delta', q_0, F')$:

1. For each state $q \in Q$ and symbol $a \in \Sigma$, compute:

$$\delta'(q, a) = \text{E-closure}\Big(\delta\big(\text{E-closure}(q), a\big)\Big) \tag{11}$$

2. Define the new set of final states F':

$$F' = \{q \in Q \mid \text{E-closure}(q) \cap F \neq \emptyset\} \tag{12}$$

2.2 Subset Construction Algorithm (NFA to DFA Conversion)

To convert an NFA $M = (Q_N, \Sigma, \delta_N, q_0, F_N)$ into an equivalent DFA $M_D = (Q_D, \Sigma, \delta_D, q_D, F_D)$:

1. $q_D = [q_0]$ (or E-closure(q_0) if converting from ϵ-NFA).
2. Set $Q_D = \{q_D\}$, unmarked.
3. While there is an unmarked composite state $U = [q_1, q_2, \ldots, q_k]$ in Q_D:
   - Mark U.
   - For each input symbol $a \in \Sigma$:

$$V = \bigcup_{i=1}^{k} \delta_N(q_i, a) \tag{13}$$

   - If $V \neq \emptyset$ and $V \notin Q_D$, add V as an unmarked composite state to Q_D.
   - Define $\delta_D(U, a) = V$.
4. $F_D = \{U \in Q_D \mid U \cap F_N \neq \emptyset\}$.

2.3 DFA Minimization (Myhill-Nerode / Box-Table Filling Method)

Minimization converts a DFA into an equivalent DFA with the minimum number of states.

Algorithm: Table Filling (Box) Method for DFA Minimization

Step 1: Remove all unreachable states from the initial state q_0.
Step 2: Draw a triangular grid for all pairs of states (p, q) where $p \neq q$.
Step 3 (Base Case): Mark every pair (p, q) with a tick '✓' (or 1) if one state is in F and the other is not ($p \in F, q \notin F$ or $p \notin F, q \in F$).
Step 4 (Inductive Pass): Repeat until no new pairs can be marked:

- For an unmarked pair (p, q), check if for any $a \in \Sigma$, the transition pair $(\delta(p, a), \delta(q, a))$ is already marked.
- If $(\delta(p, a), \delta(q, a))$ is marked, mark (p, q).

Step 5: Any unmarked pair (p, q) signifies that state p and state q are **equivalent** ($p \equiv q$). Combine all equivalent states into single block states.

2.4 Regular Expressions (RE)

A **Regular Expression** over alphabet Σ represents a regular language recursively:

2.4.1 Formal Base Inductive Definition

1. **Base Cases:**
   - $\emptyset$ is an RE denoting the empty language $\emptyset$.
   - ϵ is an RE denoting the language $\{\epsilon\}$.
   - For every $a \in \Sigma$, **a** is an RE denoting the language $\{a\}$.
2. **Inductive Steps:** If r_1 and r_2 are REs denoting languages $L(r_1)$ and $L(r_2)$:
   - $(r_1 + r_2)$ or $(r_1 \mid r_2)$ denotes $L(r_1) \cup L(r_2)$ (Union/Choice).
   - $(r_1 \cdot r_2)$ denotes $L(r_1)L(r_2)$ (Concatenation).
   - (r_1^*) denotes $(L(r_1))^*$ (Kleene Closure).

2.4.2 Algebraic Laws for Regular Expressions

$$r + s = s + r \quad \text{(Commutativity of Union)} \tag{14}$$
$$(r + s) + t = r + (s + t) \quad \text{(Associativity of Union)} \tag{15}$$
$$(rs)t = r(st) \quad \text{(Associativity of Concatenation)} \tag{16}$$
$$r(s + t) = rs + rt \quad \text{(Left Distributivity)} \tag{17}$$
$$(r + s)t = rt + st \quad \text{(Right Distributivity)} \tag{18}$$
$$r + \emptyset = r, \quad r\epsilon = \epsilon r = r \tag{19}$$
$$r + r = r \quad \text{(Idempotence)} \tag{20}$$
$$(r^*)^* = r^* \tag{21}$$
$$\emptyset^* = \epsilon, \quad \epsilon^* = \epsilon \tag{22}$$
$$(a + b)^* = (a^*b^*)^* = (a^* + b^*)^* = a^*(ba^*)^* \tag{23}$$

2.5 Arden's Theorem

Arden's Theorem Statement & Proof

Statement: Let P and Q be two regular expressions over Σ. If P does not contain the null string ϵ (i.e., $\epsilon \notin L(P)$), then the equation:

$$R = Q + RP \tag{24}$$

has a **unique solution** given by:

$$R = QP^* \tag{25}$$

Proof of Existence: Substitute $R = QP^*$ into $R = Q + RP$:

$$\text{R.H.S.} = Q + (QP^*)P = Q(\epsilon + P^*P) = QP^* = \text{L.H.S.} \tag{26}$$

Proof of Uniqueness: Substitute $R = Q + RP$ recursively into itself:

$$R = Q + (Q + RP)P = Q + QP + RP^2 \tag{27}$$

$$= Q + QP + QP^2 + \cdots + QP^k + RP^{k+1} \tag{28}$$

$$= Q(\epsilon + P + P^2 + \cdots + P^k) + RP^{k+1} \tag{29}$$

As $k \to \infty$, since $\epsilon \notin P$, the term RP^{k+1} does not contribute to strings of finite length. Hence $R = QP^*$.

Solving FA to RE using Arden's Theorem

Problem: Convert the following transition system into a Regular Expression:

- $q_1 = q_1 0 + \epsilon$
- $q_2 = q_1 1 + q_2 1$
- Final state is q_2.

Solution:

1. Apply Arden's Theorem to q_1: $q_1 = \epsilon + q_1 0 \implies q_1 = \epsilon \cdot 0^* = 0^*$.
2. Substitute q_1 into equation for q_2:

$$q_2 = (0^*1) + q_2 1 \tag{30}$$

3. Apply Arden's Theorem ($R = q_2, Q = 0^*1, P = 1$):

$$q_2 = (0^*1)1^* = 0^*1^+ \tag{31}$$

Final RE: $r = 0^*1^+$.

3 Module 3: Regular Languages & Pumping Lemma

3.1 Proving Languages Non-Regular

Not all languages are regular. Languages requiring unbounded memory (such as tracking nested parentheses or matching counts $a^n b^n$) cannot be recognized by finite state machines.

3.2 Pumping Lemma for Regular Languages

Pumping Lemma Statement

Let L be a regular language. Then there exists a constant integer $p \geq 1$ (called the **pumping length**) such that for every string $w \in L$ with length $|w| \geq p$, w can be decomposed into three substrings $w = xyz$ satisfying the following three conditions:

1. $|y| \geq 1$ (the pumped substring y is non-empty),
2. $|xy| \leq p$ (the prefix xy lies within the first p symbols),
3. For all integers $k \geq 0$, the string $xy^k z \in L$.

3.2.1 Proof Strategy by Contradiction (Adversarial Structure)

To prove a language L is **not regular**:

1. Assume L is regular. Let p be the pumping length guaranteed by the Pumping Lemma.
2. Choose a specific string $w \in L$ such that $|w| \geq p$ (expressed in terms of p).
3. Consider all possible split combinations $w = xyz$ satisfying $|xy| \leq p$ and $|y| \geq 1$.
4. Select an integer $k \geq 0$ (often $k = 0$ or $k = 2$) such that $xy^k z \notin L$.
5. This produces a contradiction, proving that L is not regular.

Proof Example 1: L

Proof:

1. Assume L is regular with pumping length p.
2. Choose string $w = a^p b^p \in L$. Length $|w| = 2p \geq p$.
3. By Pumping Lemma, $w = xyz$ where $|xy| \leq p$ and $|y| \geq 1$.
4. Since $|xy| \leq p$, the substring xy consists entirely of a's. Thus, $y = a^m$ for some $m \geq 1$.
5. Pump with $k = 2$: $xy^2 z = a^{p+m} b^p$.
6. Since $m \geq 1$, $p + m \neq p$. The string $a^{p+m} b^p$ has more a's than b's, so $xy^2 z \notin L$.
7. Contradiction! Hence L is non-regular.

Proof Example 2: L

Proof:

1. Assume L is regular with pumping length p. Choose $w = a^{p^2}$. Length $|w| = p^2 \geq p$.
2. Let $w = xyz$ with $|xy| \leq p$ and $|y| = m \geq 1$. Thus $1 \leq m \leq p$.
3. Pump for $k = 2$: xy^2z has length $p^2 + m$.
4. Evaluate bounds:
$$p^2 < p^2 + m \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2 \tag{32}$$
5. The length $p^2 + m$ lies strictly between two consecutive perfect squares p^2 and $(p+1)^2$.
6. Therefore, $xy^2z \notin L$. Contradiction! L is non-regular.

3.3 Closure Properties of Regular Languages

Regular languages are closed under a wide variety of operations.

Table 4: **Closure Properties of Regular Languages**

Operation	Closed?	Proof Sketch / Construction
Union ($L_1 \cup L_2$)	Yes	NFA parallel construction or Product DFA.
Intersection ($L_1 \cap L_2$)	Yes	Product Automaton $M = (Q_1 \times Q_2, \ldots, F_1 \times F_2)$.
Complement ($\overline{L}$)	Yes	Swap final and non-final states in a complete DFA.
Difference ($L_1 \setminus L_2$)	Yes	$L_1 \setminus L_2 = L_1 \cap \overline{L_2}$.
Concatenation (L_1L_2)	Yes	Connect final states of M_1 to start state of M_2 via ϵ.
Kleene Star (L^*)	Yes	Add new start/accept state with ϵ-loops.
Reversal (L^R)	Yes	Reverse all transition arrows; swap initial and final states.
Homomorphism ($h(L)$)	Yes	Replace input symbols a with strings $h(a)$ in RE/NFA.
Inverse Homomorphism	Yes	Define $\delta_{h^{-1}}(q, a) = \hat{\delta}(q, h(a))$.

3.4 Myhill-Nerode Theorem

Myhill-Nerode Theorem Overview

An equivalence relation R on Σ^* is a **right invariant equivalence relation** if $xRy \implies xzRyz$ for all $z \in \Sigma^*$.

- **Theorem:** A language L is regular if and only if the equivalence relation R_L (defined as $xR_Ly \iff \forall z \in \Sigma^*, xz \in L \leftrightarrow yz \in L$) has a **finite number of equivalence classes** (finite index).
- The number of equivalence classes of R_L equals the minimum number of states in a minimal DFA for L.

4 Module 4: Context-Free Grammars (CFG) & Languages

4.1 Formal Definition of Context-Free Grammar (CFG)

A Context-Free Grammar (CFG) is a 4-tuple:

$$G = (V, T, P, S) \tag{33}$$

where:

- V is a finite set of non-terminal symbols (variables).
- T is a finite set of terminal symbols ($V \cap T = \emptyset$).
- P is a finite set of production rules of the form $A \to \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.
- $S \in V$ is the start symbol.

4.2 Derivations and Parse Trees

4.2.1 Derivation Types

- **Leftmost Derivation (LMD):** At each step, the leftmost non-terminal variable is replaced first.
- **Rightmost Derivation (RMD):** At each step, the rightmost non-terminal variable is replaced first.
- **Sentential Form:** Any string $\alpha \in (V \cup T)^*$ derived from S ($S \Rightarrow^* \alpha$).

4.2.2 Parse Tree (Derivation Tree)

A parse tree is an ordered rooted tree representing the hierarchical structure of a derivation:

- Root is labeled with the start symbol S.
- Interior nodes are labeled with variables V.
- Leaf nodes are labeled with terminals T or ϵ.
- Concatenation of leaves from left to right forms the **yield** of the tree.

4.3 Ambiguity in Grammars

A CFG G is **ambiguous** if there exists at least one string $w \in L(G)$ that possesses:

- Two or more distinct leftmost derivations (LMDs), OR
- Two or more distinct rightmost derivations (RMDs), OR
- Two or more distinct parse trees.

Ambiguous Expression Grammar & Disambiguation

Ambiguous Grammar:

$$E \to E + E \mid E \times E \mid (E) \mid \text{id} \tag{34}$$

The string $\text{id} + \text{id} \times \text{id}$ has two parse trees depending on operator precedence.
Disambiguated Unambiguous Grammar (Enforcing Precedence $\times > +$):

$$E \to E + T \mid T \tag{35}$$
$$T \to T \times F \mid F \tag{36}$$
$$F \to (E) \mid \text{id} \tag{37}$$

4.4 Simplification of Context-Free Grammars

Cleaning a CFG involves three sequential phases:

3-Step CFG Simplification Procedure

Phase 1: Elimination of Useless Symbols

1. **Eliminate Non-Generating Symbols:** Find $V_G = \{A \in V \mid A \Rightarrow^* w, w \in T^*\}$. Remove variables not in V_G.
2. **Eliminate Non-Reachable Symbols:** Find $V_R = \{X \in V \cup T \mid S \Rightarrow^* \alpha X \beta\}$. Remove symbols not in V_R.

Phase 2: Elimination of Null (ϵ) Productions ($A \to \epsilon$)

1. Find set of **Nullable Variables**: $N = \{A \in V \mid A \Rightarrow^* \epsilon\}$.
2. For every rule $A \to \alpha$, add new rules formed by replacing subsets of nullable variables in α with ϵ.
3. Delete all $A \to \epsilon$ rules (unless $\epsilon \in L(G)$, then keep $S \to \epsilon$).

Phase 3: Elimination of Unit Productions ($A \to B$)

1. For each variable A, find the unit closure set $U(A) = \{B \in V \mid A \Rightarrow^* B\}$.
2. For each $B \in U(A)$ and non-unit production $B \to \alpha$, add rule $A \to \alpha$.
3. Delete all unit productions $A \to B$.

4.5 Normal Forms for CFGs

4.5.1 Chomsky Normal Form (CNF)

A CFG is in **Chomsky Normal Form (CNF)** if every production rule is of the form:

$$A \to BC \quad \text{or} \quad A \to a \tag{38}$$

where $A, B, C \in V$ and $a \in T$.

Algorithm: Conversion of CFG to CNF

Step 1: Simplify the grammar (Remove useless symbols, ϵ-productions, unit productions).
Step 2: For any production containing terminals alongside variables or multiple terminals (e.g., $A \to aB$), introduce new variable $X_a \to a$ and replace a with X_a to get $A \to X_a B$.
Step 3: For any rule with ≥ 3 variables (e.g., $A \to BCDE$), split into binary pairs by introducing cascade variables:

$$A \to BY_1, \quad Y_1 \to CY_2, \quad Y_2 \to DE \tag{39}$$

4.5.2 Greibach Normal Form (GNF)

A CFG is in **Greibach Normal Form (GNF)** if every production rule is of the form:

$$A \to a\alpha \tag{40}$$

where $a \in T$ and $\alpha \in V^*$ (a string of zero or more variables).

4.6 Pumping Lemma for Context-Free Languages (CFL)

Pumping Lemma for CFLs

Let L be a Context-Free Language. There exists a constant $n \geq 1$ such that any string $z \in L$ with $|z| \geq n$ can be written as $z = uvwxy$ such that:

1. $|vwx| \leq n$,
2. $|vx| \geq 1$ (at least one of v or x is non-empty),
3. $uv^i wx^i y \in L$ for all $i \geq 0$.

Proof: L

Proof:

1. Assume L is a CFL. Let n be the pumping length. Choose $z = a^n b^n c^n \in L$.
2. By Lemma, $z = uvwxy$ with $|vwx| \leq n$ and $|vx| \geq 1$.
3. Because $|vwx| \leq n$, the substring vwx cannot contain all three symbols a, b, c. It can contain at most two distinct symbols.
4. Thus, vx contains only a's and b's, or only b's and c's.
5. Pump $i = 2$: uv^2wx^2y increases the count of one or two symbols without increasing the third symbol.
6. The resulting string does not have equal numbers of a's, b's, and c's, so $uv^2wx^2y \notin L$.
7. Contradiction! Thus $L = \{a^n b^n c^n \mid n \geq 1\}$ is not context-free.

5 Module 5: Pushdown Automata (PDA)

5.1 Pushdown Automata Architecture

A **Pushdown Automaton (PDA)** extends a finite automaton by providing an auxiliary **Stack Memory** (Last-In, First-Out).

5.1.1 Formal Definition

A Pushdown Automaton is formally defined as a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F) \tag{41}$$

where:

- Q is a finite set of internal states.
- Σ is the finite input alphabet.
- Γ is the finite stack alphabet.
- $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to 2^{Q \times \Gamma^*}$ is the transition function.
- $q_0 \in Q$ is the initial state.
- $Z_0 \in \Gamma$ is the initial stack top symbol.
- $F \subseteq Q$ is the set of final states.

5.2 Instantaneous Description (ID)

An **Instantaneous Description (ID)** captures the complete configuration of a PDA at any step:

$$(q, w, \alpha) \in Q \times \Sigma^* \times \Gamma^* \tag{42}$$

where q is current state, w is unread input, and α is full stack content (leftmost symbol is top of stack).

5.2.1 Transition Notation

If $\delta(q, a, X)$ contains (p, γ), we write the turnstile transition move as:

$$(q, aw, X\beta) \vdash (p, w, \gamma\beta) \tag{43}$$

5.3 Language Acceptance Criteria

1. **Acceptance by Final State:**

$$L(M) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q_f, \epsilon, \gamma) \text{ for some } q_f \in F, \gamma \in \Gamma^*\} \tag{44}$$

2. **Acceptance by Empty Stack (Null Stack):**

$$N(M) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon) \text{ for any } q \in Q\} \tag{45}$$

Equivalence Theorem: $L(M)$ and $N(M)$ are computationally equivalent; any PDA accepting by final state can be converted into a PDA accepting by empty stack, and vice versa.

PDA Construction: L

Logic: Non-deterministically push input symbols onto stack for the first half w. Guess the midpoint, then pop matching symbols for w^R.
Formal Transitions (δ):

$$\delta(q_0, a, Z_0) = \{(q_0, aZ_0)\}, \quad \delta(q_0, b, Z_0) = \{(q_0, bZ_0)\} \tag{46}$$
$$\delta(q_0, a, a) = \{(q_0, aa)\}, \quad \delta(q_0, b, a) = \{(q_0, ba)\} \tag{47}$$
$$\delta(q_0, a, b) = \{(q_0, ab)\}, \quad \delta(q_0, b, b) = \{(q_0, bb)\} \tag{48}$$
$$\textit{(Guess Midpoint):} \quad \delta(q_0, \epsilon, a) = \{(q_1, a)\}, \quad \delta(q_0, \epsilon, b) = \{(q_1, b)\} \tag{49}$$
$$\textit{(Pop Phase):} \quad \delta(q_1, a, a) = \{(q_1, \epsilon)\}, \quad \delta(q_1, b, b) = \{(q_1, \epsilon)\} \tag{50}$$
$$\textit{(Acceptance):} \quad \delta(q_1, \epsilon, Z_0) = \{(q_f, Z_0)\} \quad \text{where } q_f \in F. \tag{51}$$

5.4 Deterministic Pushdown Automata (DPDA)

A PDA is **Deterministic (DPDA)** if:

For any $q \in Q, a \in \Sigma \cup \{\epsilon\}, X \in \Gamma$, $|\delta(q, a, X)| \leq 1$.

If $\delta(q, \epsilon, X) \neq \emptyset$, then $\delta(q, a, X) = \emptyset$ for all $a \in \Sigma$. **Note:** DPDA is strictly less powerful than NPDA. For instance, $L = \{ww^R\}$ can be recognized by NPDA but not by DPDA.

5.5 Conversion of CFG to PDA

Given a CFG $G = (V, T, P, S)$ in Greibach Normal Form (GNF): Construct single-state PDA $M = (\{q\}, T, V \cup T, \delta, q, S, \emptyset)$:

1. For each rule $A \to a\alpha \in P$, add: $\delta(q, a, A) = \{(q, \alpha)\}$.
2. For each terminal $a \in T$, add: $\delta(q, a, a) = \{(q, \epsilon)\}$.

6 Module 6: Turing Machines (TM) & Undecidability

6.1 Turing Machine Model

A **Turing Machine (TM)** is the most powerful abstract model of computation, consisting of a finite control, a tape scanner head, and an infinite tape divided into discrete cells.

6.1.1 Formal Definition

A Turing Machine is formally defined as a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F) \tag{52}$$

where:

- Q is a finite set of internal states.
- Σ is the input alphabet ($B \notin \Sigma$).
- Γ is the tape alphabet ($\Sigma \subset \Gamma, B \in \Gamma$).
- $B \in \Gamma$ is the blank symbol.
- $q_0 \in Q$ is the start state.
- $F \subseteq Q$ is the set of final/halt states.
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, N\}$ is the transition function mapping (state, scanned symbol) to (next state, replacement symbol, head move Left/Right/No move).

Turing Machine Solved Example: Language L

Execution Strategy:

1. Scan left to right: Mark the leftmost a with X. Continue right to find leftmost b, mark with Y. Continue right to find leftmost c, mark with Z.
2. Move tape head left until an X is encountered. Move right to the next available a.
3. Repeat process. If all a's, b's, and c's are systematically matched and replaced by X, Y, Z without leftover symbols, HALT and ACCEPT.

Transition Function Subset (δ):

$$\delta(q_0, a) = (q_1, X, R) \quad \text{(Mark } a \text{ with } X\text{)} \tag{53}$$
$$\delta(q_1, a) = (q_1, a, R), \quad \delta(q_1, Y) = (q_1, Y, R) \tag{54}$$
$$\delta(q_1, b) = (q_2, Y, R) \quad \text{(Mark } b \text{ with } Y\text{)} \tag{55}$$
$$\delta(q_2, b) = (q_2, b, R), \quad \delta(q_2, Z) = (q_2, Z, R) \tag{56}$$
$$\delta(q_2, c) = (q_3, Z, L) \quad \text{(Mark } c \text{ with } Z\text{, reverse direction)} \tag{57}$$
$$\delta(q_3, \text{unmarked}) = (q_3, \text{unmarked}, L) \quad \text{until } X \text{ found.} \tag{58}$$

6.2 Variants of Turing Machines

All the following variants are **computationally equivalent** to a standard single-tape Turing Machine:

- **Multi-tape Turing Machine:** Has k independent tapes and k heads.
- **Multi-track Turing Machine:** Single head reads k tracks on a single tape cell simultaneously.
- **Two-Way Infinite Tape TM:** Tape extends infinitely both left and right.
- **Nondeterministic Turing Machine (NDTM):** $\delta : Q \times \Gamma \to 2^{Q \times \Gamma \times \{L,R\}}$. Equivalent in computing power to deterministic TM.
- **Universal Turing Machine (UTM):** A TM capable of simulating any arbitrary TM M given its encoded description $\langle M \rangle$ and input w.

6.3 Church-Turing Thesis

Church-Turing Thesis

Statement: Any algorithmically solvable problem or effective computation can be carried out by a Turing Machine.
The Church-Turing thesis establishes the Turing machine as the ultimate benchmark for mechanical computation.

6.4 Halting Problem of Turing Machines

Halting Problem Undecidability Proof

Problem Statement (H): Given a Turing Machine M and an input string w, determine whether M will eventually halt when run on input w or run forever.
Theorem: The Halting Problem is **Undecidable**.
Proof by Contradiction (Diagonalization):

1. Assume there exists a decider TM H that solves the halting problem:

$$H(\langle M\rangle, w) = \begin{cases} \text{Accept} & \text{if } M \text{ halts on } w \\ \text{Reject} & \text{if } M \text{ loops forever on } w \end{cases} \tag{59}$$

2. Construct a new TM D that takes a description $\langle M\rangle$ as input and invokes $H(\langle M\rangle, \langle M\rangle)$:
   - If H says "Accept" (M halts on $\langle M\rangle$), D intentionally enters an **infinite loop**.
   - If H says "Reject" (M loops on $\langle M\rangle$), D **halts immediately** and accepts.
3. Now run D on its own description $\langle D\rangle$:
   - If D halts on $\langle D\rangle \implies H(\langle D\rangle, \langle D\rangle)$ accepts $\implies D$ loops forever!
   - If D loops forever on $\langle D\rangle \implies H(\langle D\rangle, \langle D\rangle)$ rejects $\implies D$ halts!
4. In both cases, we reach a self-contradictory paradox (D halts if and only if D loops forever).
5. Thus, the assumption that H exists must be false. The Halting Problem is **undecidable**.

6.5 Rice's Theorem

Rice's Theorem

Statement: Any non-trivial semantic property of the language recognized by a Turing Machine is undecidable.
A property P of Recursively Enumerable languages is:

- **Semantic:** Depends on $L(M)$, not the internal encoding of M.
- **Non-trivial:** There exists at least one TM whose language has property P, and at least one TM whose language does not have property P.

Implication: Questions such as "Is $L(M)$ empty?", "Is $L(M)$ finite?", or "Is $L(M)$ regular?" are all undecidable.

6.6 Post Correspondence Problem (PCP)

The **Post Correspondence Problem (PCP)** is a prominent undecidability result outside automata theory.

PCP Formulation Solved Instance

Problem Statement: Given two lists of strings $A = (w_1, w_2, \ldots, w_k)$ and $B = (x_1, x_2, \ldots, x_k)$ over Σ, does there exist a sequence of indices $i_1, i_2, \ldots, i_m$ $(m \geq 1)$ such that:

$$w_{i_1} w_{i_2} \ldots w_{i_m} = x_{i_1} x_{i_2} \ldots x_{i_m} \tag{60}$$

Example Instance: Let $A = (10, 011, 101)$, $B = (101, 11, 011)$.

Index i	List A (w_i)	List B (x_i)
1	10	101
2	011	11
3	101	011

Solution Sequence: $i = (3, 2, 3, 1)$

- A concatenation: $w_3 w_2 w_3 w_1 = (101)(011)(101)(10) = 10101110110$
- B concatenation: $x_3 x_2 x_3 x_1 = (011)(11)(011)(101) = 01111011101$

Since $10101110110 \neq 01111011101$, this sequence does not match. Searching for a matching sequence yields a solution if one exists, but PCP is undecidable in general!

6.7 Recursive and Recursively Enumerable Languages

Table 5: **Classification of Languages in Computability Theory**

Property	Recursive Language (Decidable)	Recursively Enumerable (RE)
Turing Machine Behavior	TM halts on ALL inputs (Accept or Reject).	TM halts and accepts if $w \in L$; may loop forever if $w \notin L$.
Decision Status	Decidable language.	Turing-recognizable language.
Closure under Complement	Closed under complement ($\overline{L}$ is Recursive).	Not closed under complement (if L and $\overline{L}$ are RE $\implies$ L is Recursive).

7 University Exam Revision Solved Question Bank

This section consolidates top-tier university exam questions across all modules with detailed answer frameworks.

7.1 Short Answer Concept Clarification Questions (5 Marks Each)

Q1: State the state equivalence theorem and explain how distinguishable states are identified.

Solution Framework

Two states $p, q \in Q$ are equivalent ($p \equiv q$) iff for all strings $w \in \Sigma^*$, $\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$. If there exists at least one string w such that $\hat{\delta}(p, w) \in F$ and $\hat{\delta}(q, w) \notin F$, the states are distinguishable by string w.

Q2: Construct a regular expression for strings over $\Sigma = \{0, 1\}$ having even number of 0's followed by odd number of 1's.

Solution

Even number of 0's: $(00)^*$
Odd number of 1's: $1(11)^*$ or $(11)^*1$
Concatenation: $r = (00)^*1(11)^*$

Q3: Differentiate between DPDA and NPDA with suitable language examples.

Solution

DPDA has at most one move per configuration and cannot accept languages with inherent non-determinism like $L = \{ww^R \mid w \in \{a, b\}^*\}$. NPDA uses non-deterministic guesses to identify midpoint of ww^R or parse ambiguous CFGs.

Q4: Explain the Halting Problem and state why it is undecidable.

Solution

The halting problem asks if an arbitrary TM M halts on input w. It is proven undecidable using Cantor's diagonalization technique: assuming a decider H leads to a contradictory machine D that halts iff it loops.

7.2 Long Analytical Construction Problems (10 Marks Each)

Q1:, start=5 Convert the following CFG to Chomsky Normal Form (CNF):

$$S \to aAbB \mid ab, \quad A \to aA \mid a, \quad B \to bB \mid b \tag{61}$$

Step-by-Step CNF Solution

Step 1: Grammar is already simplified (no null or unit productions).
Step 2: Replace terminals in length ≥ 2 body: Introduce $X \to a$ and $Y \to b$. New rules: $S \to XAYB \mid XY, \quad A \to XA \mid a, \quad B \to YB \mid b$
Step 3: Break productions of length > 2: For $S \to XAYB$: $S \to XC_1, \quad C_1 \to AC_2, \quad C_2 \to YB$
Final CNF Grammar:

$$S \to XC_1 \mid XY \tag{62}$$
$$C_1 \to AC_2 \tag{63}$$
$$C_2 \to YB \tag{64}$$
$$A \to XA \mid a \tag{65}$$
$$B \to YB \mid b \tag{66}$$
$$X \to a, \quad Y \to b \tag{67}$$

Q2:, start=5 Design a Turing Machine to perform 1's complement of a binary number.

Solution

Tape Alphabet $\Gamma = \{0, 1, B\}$, Start State q_0. Transitions:

$$\delta(q_0, 0) = (q_0, 1, R) \quad \text{(Flip 0 to 1 and move right)} \tag{68}$$
$$\delta(q_0, 1) = (q_0, 0, R) \quad \text{(Flip 1 to 0 and move right)} \tag{69}$$
$$\delta(q_0, B) = (q_f, B, N) \quad \text{(Halt upon reaching Blank)} \tag{70}$$

8 Summary Formula Cheat Sheet

Table 6: **Master Automata Theory Reference Summary**

Concept / Model	Key Formula / Tuple	Important Property / Identity
DFA	$M = (Q, \Sigma, \delta, q_0, F)$	$\delta : Q \times \Sigma \to Q$ (Deterministic)
NFA	$M = (Q, \Sigma, \delta, q_0, F)$	$\delta : Q \times \Sigma \to 2^Q$ (Power Set)
Arden's Theorem	$R = Q + RP$	Unique solution $R = QP^*$ if $\epsilon \notin P$
Pumping Lemma (RL)	$w = xyz, \lvert xy\rvert \leq p, \lvert y\rvert \geq 1$	$xy^kz \in L, \forall k \geq 0$
CNF (Grammar)	$A \to BC$ or $A \to a$	Binary variable splits or single terminal
GNF (Grammar)	$A \to a\alpha$	First symbol is terminal $a \in T$
PDA	$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$	$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to 2^{Q \times \Gamma^*}$
Pumping Lemma (CFL)	$z = uvwxy, \lvert vwx\rvert \leq n, \lvert vy\rvert \geq 1$	$uv^kwx^ky \in L, \forall k \geq 0$
Turing Machine	$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$	$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, N\}$

End of Automata Theory Revision Guide

Best of luck with your Automata Theory examinations!